3IFA DW Lab: Dynamic Webpage with MongoDB and Javascript 2026

MongoDB + Javascript + NodeJS + Express

Introduction

Read carefully and follow the instructions!

The purpose of this lab is to introduce you to the creation of dynamic webpages using MongoDB, Javascript, HTML and XML. For this purpose you will use `Firefox` as a browser, `MongoDB` as the database management system, `NodeJS` and `Express` as backend technology and framework.

Use a code editor (not a text processor like MS Word, LibreOffice, Google Docs) to edit your code. Some examples : VSCode, NotePad++, NetBeans, Eclipse, vi, ...

The lab spans over 2 sessions of 4 hours each. The 2 last hours of the second session will be dedicated to a new, evaluated subject that covers the concepts seen in this statement. You will have to hand in that evaluated exercice one copy per pair (binôme, trinôme). More details to come.

A very useful site for practical work: W3Schools (reference and tutorials on XML, DTD/Schema, XSL, javascript, ...)

We advise you to take notes, for example: a "main" text or html document containing your commented answers to the questions. Number the answers and copy their statements. To "color" your pieces of code you can use the site: http://www.tohtml.com/ or style libraries in javascript.

You will need to manipulate files, install programs and run scripts in a command line environment.

- On Windows you have the Command Line tool (launch it with: `Windows` + "R" keys, then type "cmd" and `Enter`)
- On MacOS launch the `terminal` app (`Command` + space bar, then type "terminal")
- On Linux launch the `terminal` app

Read the statement and do the exercises in the order of the statement!

0. Dataset Context

We suppose that you have the MongoDB server, a MongoDB client and the `mongoimport` tool installed on your computer. If not, please refer to the MongoDB exercise statement to install them.

You find here the velov2026.json file with the data you will work on. Download (mouse right button : save link target as) and import it in your `MongoDB` server to the `velov2026` collection in the `20263IFMongoLab` database.

You are working with the `velov2026` collection. Each document represents a real-time snapshot of a Lyon Vélo'v station. Key fields include the the `name` , `address` ,

`total_stands` (nested availabilities), `geometry` (GeoJSON), and `status` ...

Take your time to study the collection structure and data. You have an example snippet here:

DATABASE LAYER

Part 1: MongoDB Queries

Exploring the data

Write MongoDB queries for the following questions. Consider creating indexes when necessary. Project results to the necessary fields:

1. **Size:** Calculate the number of velov stations in the collection.
2. **Filtering:** Retrieve all stations in `Villeurbanne`.
3. **Availability:** Find all `OPEN` stations with at least 1 mechanical bike.
4. **Payment:** List the `name` and `address` of stations offering `banking`.
5. **Capacity:** Sort and return the top 5 stations by `bike_stands`.
6. **Alerts:** Find `OPEN` stations with 0 available bikes, sorted by `pole`.
7. **Projection:** Find Lyon 5e stations; return only `name` and `available_bikes`.
8. **Statistics:** Use `$group` to calculate the total number of `available_bikes` and the average capacity per `commune`.

9. **Logic:** Find stations where `electricalBikes` make up > 50% of `available_bikes`. Use the `$expr` and `$multiply` operators.
10. **Time:** Calculate the days elapsed since `last_update` for each station.
11. **Geospatial:** Find all stations within **500m** of the Confluence Darse point `[4.815, 45.743]`.

APPLICATION LAYER

Part 2: Iterative Web Development

Dynamic Web Page

The goal of this part is to build a dynamic web application to visualize the velov'geo dataset.

Create a webpage with some dynamic content.

1. **Initial webpage:** Download the example TP.html page. You will modify it in the following questions
2. **A first 2 buttons:** add a button with the text : "Blue" that changes the page background color to blue and the button text color to white, create a second button with the text "White", that changes the background color back to white and the button text to black. Create javascript functions to change the colors.
3. **Text field:** Add a text field and a button "Add text". When clicking on the button, call ajavascript function that adds the content of the text field to an ordered list below. Use

javascript and DOM methods to handle this, add @id attributes to the textfield and the list for a simpler access.

> You can debug the javascript code in the browser. Take a look at this tutorial to find out more

4. **Event handling**: Create button that makes the list from the previous question "clickable". When clicking on the list, its text is recopied in the text field. Use addEventListener for the "click" event to handle this. Iterate over the list items and add to each item and eventListener.
5. **SVG**: Download the lyon_districts.svg file next to the TP.html file. Add a button "Load svg" that loads and shows the map contained in this file.

> Don't forget to modify the `security.fileuri.strict_origin_policy` browser parameter: go to the page: `about:config`, look for the option: `security.fileuri.strict_origin_policy`, set its value to false.
>
> Use the following function to load the svg file in the html file

```
async function fetchSvgContent(url) {
    try {
        const response = await fetch(url);
        if (!response.ok) {
            throw new Error(`HTTP error! Status: ${response.status}`);
        }
        return await response.text();
    } catch (error) {
        console.error("Error fetching the SVG:", error);
        return null;
    }
}
async function initializeMap() {
    const svgData = await fetchSvgContent('lyon_districts.svg');
    if (svgData) {
```

```
            const container = document.getElementById("svgContainer");
            container.innerHTML = svgData;

        }
    }
```

6. **Dynamic SVG**: Add another button "Clickable SVG" that makes the SVG clickable. When clicking on a district, show on the page : "clicked on *{district name}*" replace *{district name}* with the value of the @id attribute of the path corresponding to it.

Web application

The goal of this part is to build a dynamic web application to visualize the velov'geo dataset.
Install NodeJS (restart the terminal after installation) and set up a simple web application. You can get help **here**.

1. **Webpage V1**. Create another route that returns only station names and communes. Modify the index.html page that calls the route and prints the list of station names with their commune in a table.
2. **Query Params:** Allow the API to filter by `?commune=` via the URL
3. **Filtering V1**: Add a dropdown list filled dynamically with the distinct communes. Filter the shown communes wy the selected commune at each changement of the selected element in the dropdown list.
4. I**ntegrating the SVG**: Show the lyon_districts.svg on the page and when clicking on a district, list the velov stations from the clicked district.

5. **Dynamic SVG**: Capture the entering (`mouseover`) and exiting (`mouseleave`) event of the mouse over the districts on the map. For the dirtict with the mouse over: change their color and display their: name, tne number of velov stations and the average number of available bikes in a table above the map
6. **MongoDB Data enrichment**: Look for downloadable data in JSON format on the website https://data.grandlyon.com/ (or elsewhere) that can complement the information of Vélo'V stations.
    1. Provide the URL that allows you to download the data (download the GeoJSON format) and explain how you think it can complement the MongoDB collection of Vélo'V stations.
    2. Download the data and import it into MongoDB, simplify it to have more than one document. Describe your procedure.
    3. Provide a MongoDB query that combines the new information with the velov station data. Describe your method. Provide: query, results and explanation.
7. Reuse the additional dataset you have downloaded for the previous question and integrate it in the webpage

Bonus

Normal Bonus: Propose and interesting visualization / interaction for the velov stations involving a complex MongoDB query. You can integrate libraries like LeafletJs or 3Djs or others.

Extra Bonus: Subject here.